Principios de Arquitectura de Software · Taller en grupo

Arquitectura Event-Driven multilenguaje con RabbitMQ y Docker

4 productores · 4 consumidores · 4 lenguajes · 3 escenarios de despliegue

Python Java Node.js Go

David Esteban Suarez Lozano · 2221909
Thomas Alejandro Perez Rojas · 2221906
Jesus David Santodomingo Carrascal · 2221922

Agenda

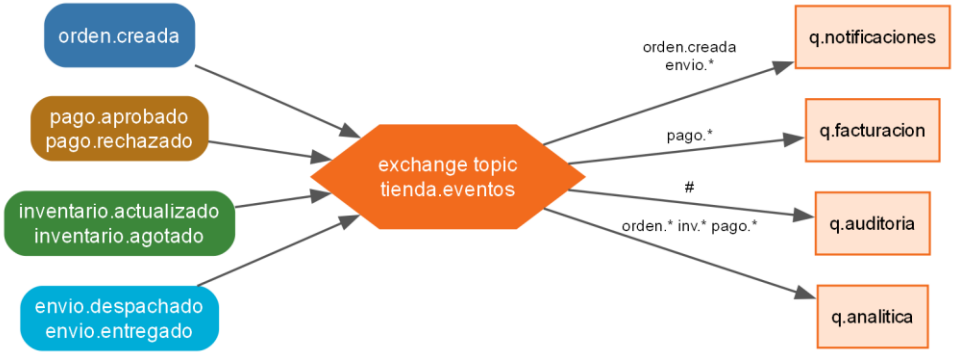
1. **Arquitectura**
   Estilo event-driven y componentes
2. **Conexiones**
   AMQP: exchange, bindings y colas
3. **Docker Compose**
   Dockerfiles y un compose por nodo
4. **El código**
   Productores y consumidores
5. **Roles por escenario**
   Qué corre en cada nodo
6. **Funcionamiento**
   Demo y evidencias

1 · Arquitectura

Event-driven con un broker de mensajes

Arquitectura event-driven

- Productores publican eventos de dominio sin saber quién los consume
- RabbitMQ es el único punto de contacto (bus de eventos)
- Consumidores reaccionan a los eventos que les interesan
- Comunicación asíncrona y desacoplada: se agregan o mueven servicios sin tocar los demás



Caso de uso: eventos de una tienda en línea

Componentes del sistema

PRODUCTORES

Órdenes

Python · pika

orden.creada

Pagos

Java · amqp-client

pago.aprobado / rechazado

Inventario

Node.js · amqplib

inventario.actualizado / agotado

Envíos

Go · amqp091-go

envio.despachado / entregado

CONSUMIDORES

Notificaciones

Python

Correos y SMS al cliente

Facturación

Java

Emite facturas o anula órdenes

Auditoría + UI

Node.js

Registra todo y lo muestra en web :8080

Analítica

Go

Métricas: ventas, pagos, stock

2 · Conexiones

AMQP 0-9-1 sobre TCP :5672

Cómo viaja un evento

1

Conexión

TCP al broker :5672, usuario admin, nombre servicio@nodo

2

Publish

Al exchange tienda.eventos con routing key = tipo de evento

3

Exchange topic

Compara la routing key con los bindings de cada cola

4

Cola durable

Guarda el mensaje persistente hasta que se consuma

5

Consume + ack

El consumidor procesa (prefetch) y confirma con ack manual

Cola	Bindings (routing keys)
q.notificaciones	orden.creada · envio.*
q.facturacion	pago.aprobado · pago.rechazado
q.auditoria	# (todos los eventos)
q.analitica	orden.* · inventario.* · pago.*

* = exactamente una palabra
\# = cero o más palabras
Un evento puede llegar a varias colas a la vez (pub/sub)

El evento: un contrato JSON común

Mensaje (application/json, delivery_mode=2)

```
{
  "id": "6f1c…-uuid",
  "tipo": "pago.aprobado",    ← routing key
  "origen": "productor-java",
  "lenguaje": "Java",
  "nodo": "esc3-nodo1",       ← nodo origen
  "timestamp": "2026-10-28T14:31:56Z",
  "datos": { "orden_id": 4576,
             "monto": 1498.11,
             "metodo": "pse" }
}
```

- JSON es el idioma común: Python, Java, Node.js y Go lo leen sin acoplarse entre sí
- Mensajes persistentes en colas durables: sobreviven a un reinicio del broker
- El campo nodo y el nombre de la conexión (servicio@nodo) permiten rastrear de qué máquina viene cada evento

3 · Docker Compose

Un contenedor por servicio · un compose por nodo

Dockerfiles multi-stage

productores/go-envios/Dockerfile

```
# Etapa 1: compilar el binario de Go
FROM golang:1.23-alpine AS build
WORKDIR /src
COPY go.mod ./
COPY *.go ./
RUN go mod tidy && CGO_ENABLED=0 go build -o /app .

# Etapa 2: imagen mínima con solo el binario
FROM alpine:3.20
COPY --from=build /app /app
CMD ["/app"]
```

- Cada servicio se compila dentro de Docker: en los PCs solo hace falta Docker Desktop
- Java (Maven → JRE) y Go (golang → alpine) usan multi-stage: imágenes livianas
- Python y Node.js: imagen slim + dependencias (pika, amqplib)

Escenario 1: un solo docker-compose.yml

escenario1-centralizado/docker-compose.yml

```
services:
  rabbitmq:
    image: rabbitmq:3-management
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin123
    ports: ["5672:5672", "15672:15672"]
    healthcheck:
      test: ["CMD","rabbitmq-diagnostics","-q",
             "check_port_connectivity"]

  productor-python:
    build: ../productores/python-ordenes
    environment: *rabbit-env   # RABBITMQ_HOST: rabbitmq
    depends_on:
      rabbitmq: { condition: service_healthy }
  # … 3 productores y 4 consumidores más
```

- 9 contenedores en la misma red Docker
- Los servicios encuentran al broker por nombre: rabbitmq
- healthcheck + depends_on: los servicios esperan a que el broker esté listo
- Anclas YAML (x-rabbit-env) para no repetir la configuración

Escenarios 2 y 3: un compose por nodo

Escenario	Archivo	Contenido
2	nodo1-servicios/docker-compose.yml	4 productores + 4 consumidores
2	nodo2-broker/docker-compose.yml	RabbitMQ
3	nodo1/docker-compose.yml	Python + Java (prod. y cons.)
3	nodo2/docker-compose.yml	Node.js + Go (prod. y cons.)
3	nodo3-broker/docker-compose.yml	RabbitMQ

.env → variables

```
# .env del nodo de servicios
BROKER_IP=10.6.62.18
BROKER_PORT=5672
RABBITMQ_USER=admin
RABBITMQ_PASS=admin123

# docker-compose.yml
RABBITMQ_HOST: ${BROKER_IP}
NODO: esc3-nodo1
```

Mismas imágenes y mismo código en los 3 escenarios: solo cambia dónde corre cada contenedor y la IP del broker

El broker expone 5672 (AMQP) y 15672 (panel) a la LAN

4 · El código

El mismo patrón en 4 lenguajes

Productor (Python + pika)

productores/python-ordenes/producer.py

```
canal.exchange_declare(exchange="tienda.eventos",
                       exchange_type="topic", durable=True)
while True:
    evento = {"id": str(uuid.uuid4()),
              "tipo": "orden.creada",
              "origen": "productor-python",
              "nodo": NODO,
              "datos": {...}}
    canal.basic_publish(
        exchange="tienda.eventos",
        routing_key=evento["tipo"],
        body=json.dumps(evento),
        properties=pika.BasicProperties(
            content_type="application/json",
            delivery_mode=2))          # persistente
    conexion.sleep(INTERVALO)
```

1

Conecta con reintento

Si el broker no está listo, reintenta cada 3 s

2

Declara el exchange

Idempotente: todos lo declaran igual

3

Publica cada 3 s

routing key = tipo del evento

Consumidor (Node.js + amqplib)

consumidores/node-auditoria/consumer.js

```
const BINDINGS = ['#'];          // auditoría: todo
await canal.assertExchange(EXCHANGE, 'topic', { durable: true });
await canal.assertQueue(COLA, { durable: true });
for (const clave of BINDINGS)
  await canal.bindQueue(COLA, EXCHANGE, clave);
await canal.prefetch(20);

await canal.consume(COLA, (msg) => {
  const evento = JSON.parse(msg.content.toString());
  console.log(`<- ${evento.tipo} de ${evento.origen}`);
  emitir(evento);                // UI web :8080
  canal.ack(msg);                // ack manual
});
```

1 **Cola propia**

Cada consumidor declara su cola durable

2 **Bindings**

Define qué eventos le interesan

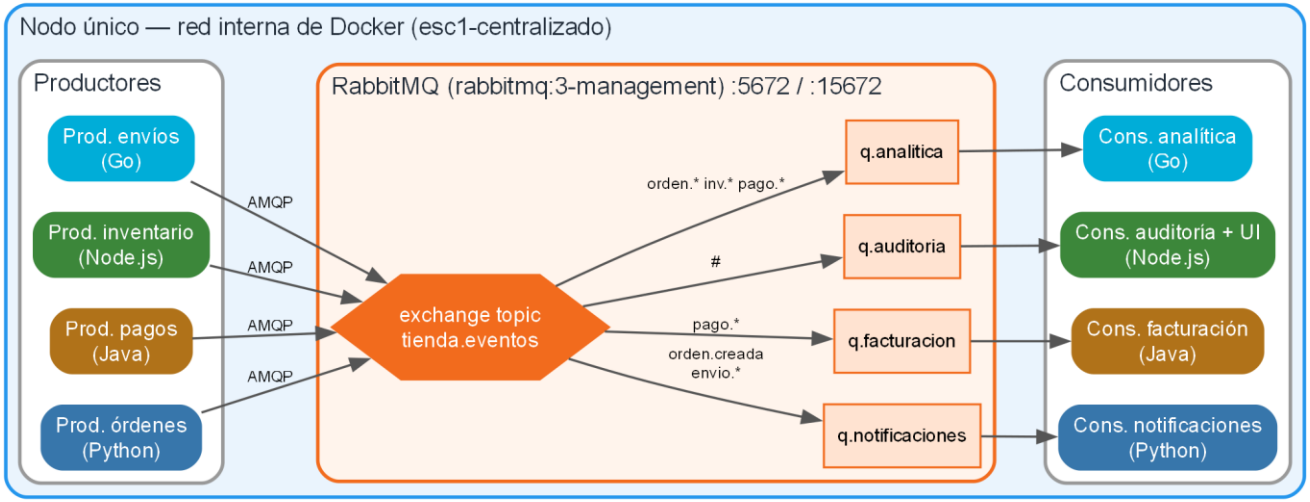
3 **prefetch + ack**

No se satura y no pierde mensajes

5 · Roles por escenario

Qué corre en cada nodo

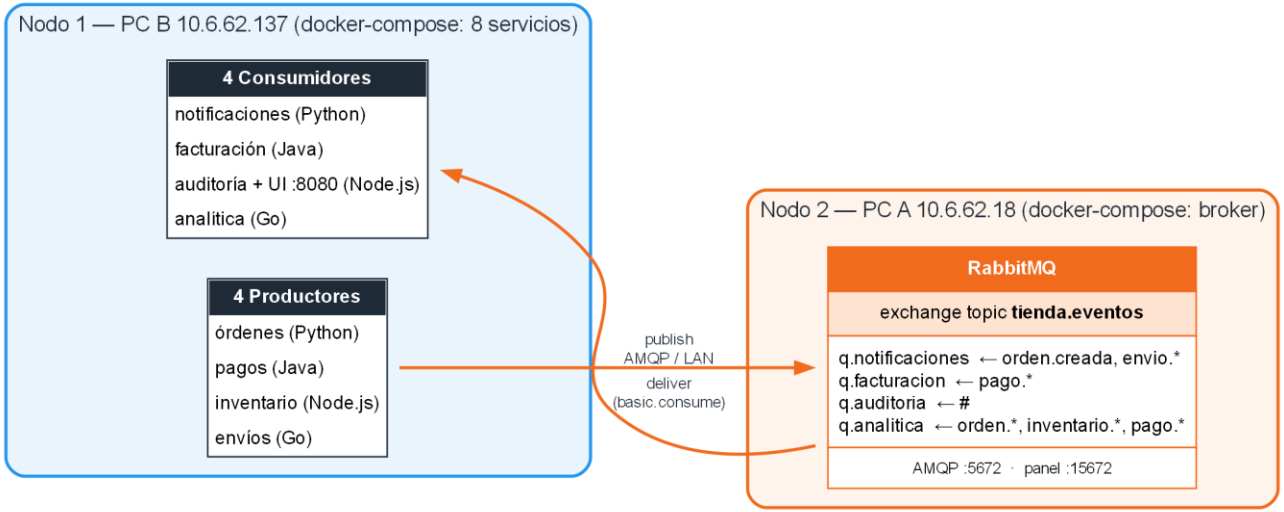
Escenario 1 · Centralizado



Nodo único

Broker + 4 productores + 4 consumidores (9 contenedores). Red interna de Docker, broker por nombre

Escenario 2 · Distribuido en 2 nodos



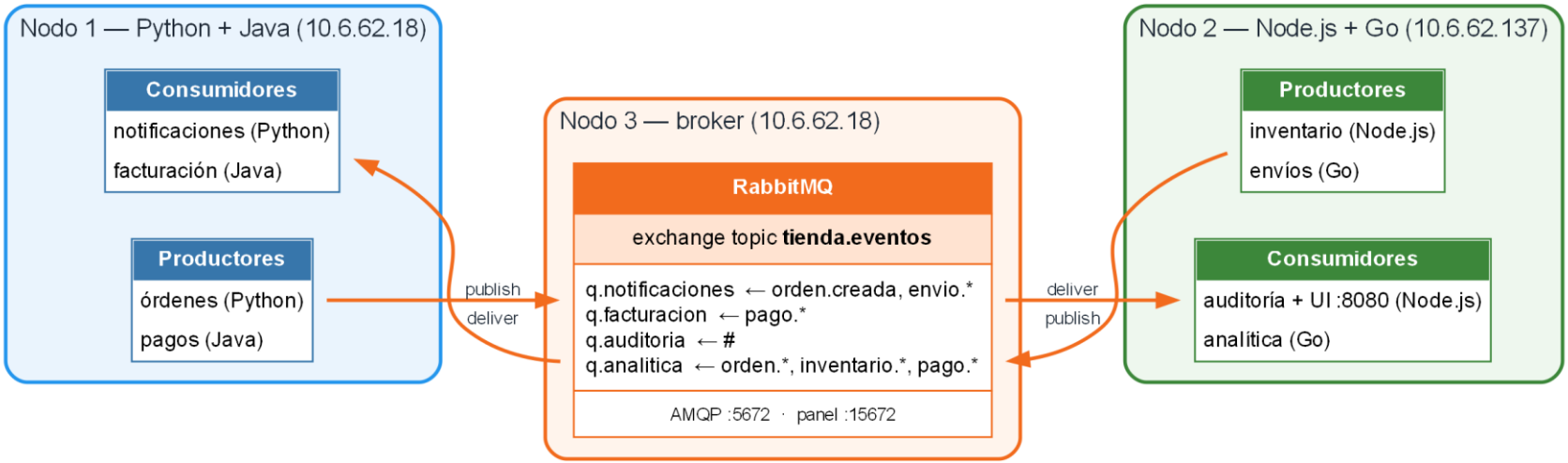
Nodo 1 · PC B (10.6.62.137)

Servicios: los 4 productores y los 4 consumidores

Nodo 2 · PC A (10.6.62.18)

Broker RabbitMQ: exchange, colas y panel de administración

Escenario 3 · Distribuido en 3 nodos



Nodo 1 · Python + Java

Prod. órdenes y pagos · Cons. notificaciones y facturación

Nodo 3 · Broker

RabbitMQ: enruta eventos entre los dos nodos de servicios

Nodo 2 · Node.js + Go

Prod. inventario y envíos · Cons. auditoría (UI) y analítica

Resumen de roles

	Broker	Productores	Consumidores	Archivos compose
Escenario 1	Nodo único	Nodo único (4)	Nodo único (4)	1
Escenario 2	Nodo 2 (PC A)	Nodo 1 (4)	Nodo 1 (4)	2
Escenario 3	Nodo 3	Nodo 1: Python, Java Nodo 2: Node.js, Go	Nodo 1: Python, Java Nodo 2: Node.js, Go	3

En los tres escenarios el broker es el único servicio al que todos se conectan: los productores y consumidores nunca se conocen entre sí.

6 · Funcionamiento

Demo en vivo y evidencias

Demo: cómo se levanta

Terminal

```
# Escenario 1 (un PC)
cd escenario1-centralizado
docker compose up --build

# Escenarios 2 y 3
# 1) PC broker
cd escenario3-tres-nodos/nodo3-broker
docker compose up -d
# 2) PCs de servicios: poner BROKER_IP en .env
cd escenario3-tres-nodos/nodo2
docker compose up --build
```

Panel RabbitMQ

http://<broker>:15672 → Connections, Exchanges, Queues

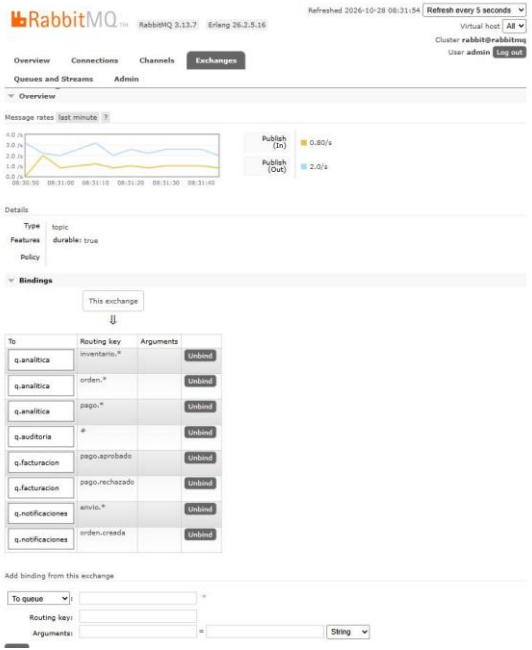
UI de eventos

http://<nodo>:8080 → eventos en vivo

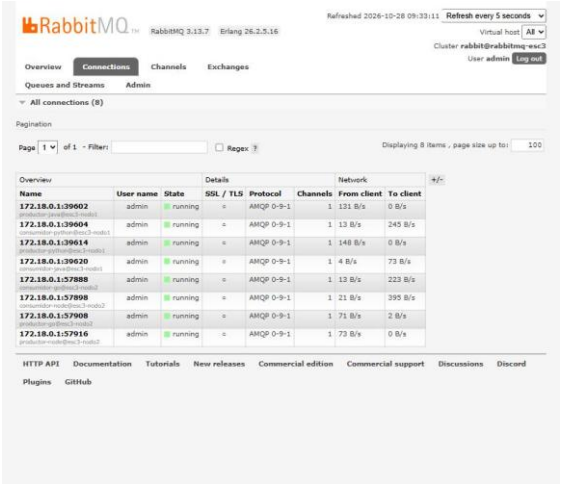
Logs

docker compose logs -f consumidor-go

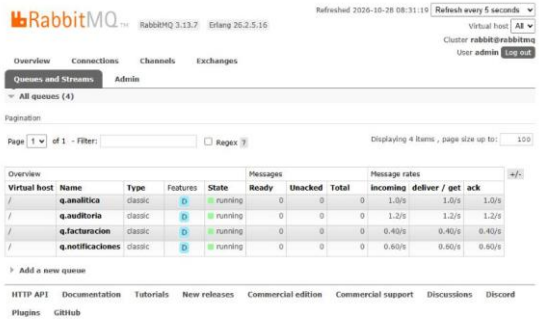
Evidencia: panel de RabbitMQ



Exchange topic + 8 bindings

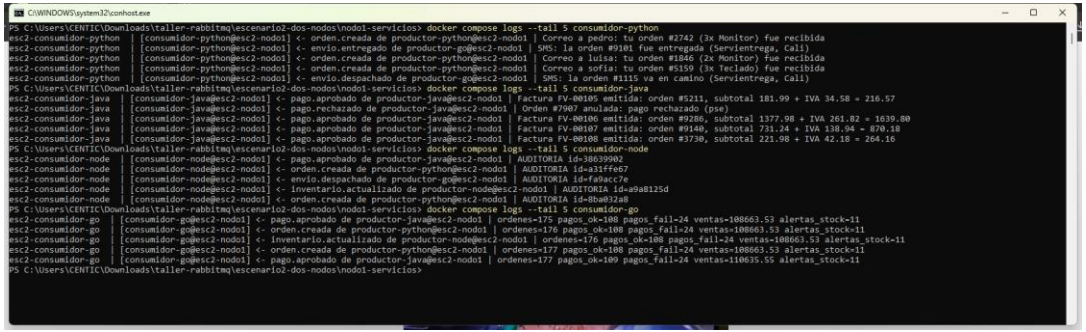


Esc. 3: 4 conexiones @nodo1 y 4 @nodo2



4 colas durables, 1 consumidor c/u

Evidencia: eventos cruzando nodos



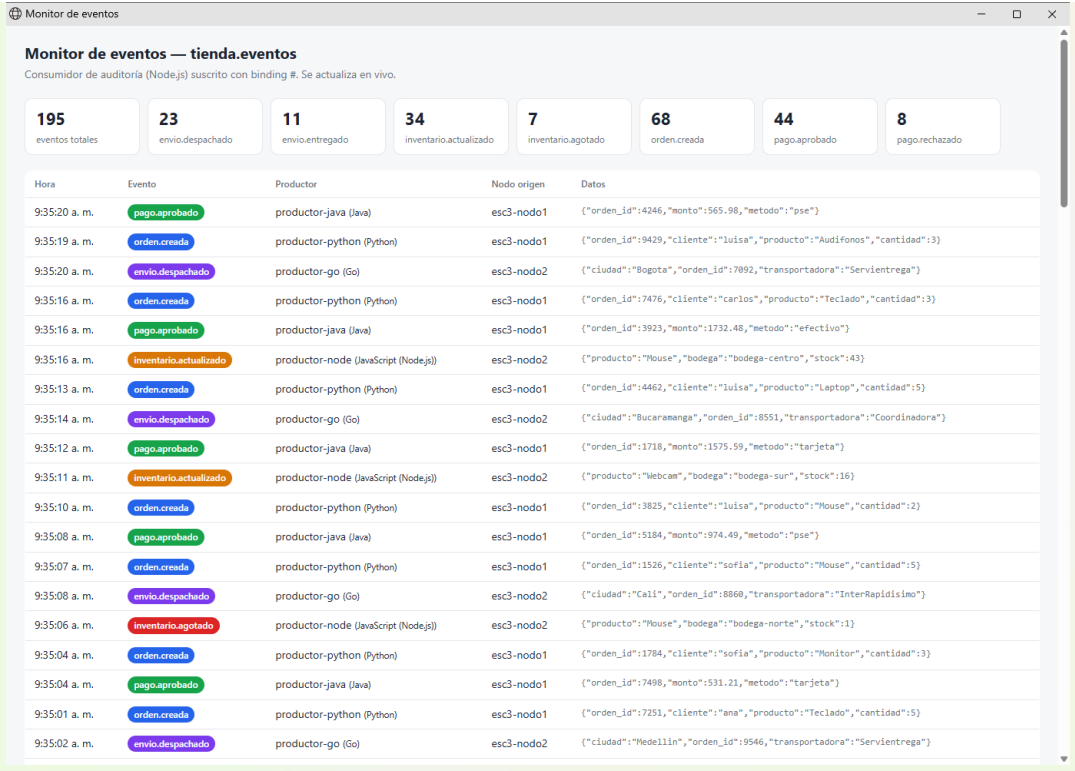
Escenario 2: los 4 consumidores del PC B procesan eventos vía el broker del PC A

Escenario 3: tráfico cruzado Nodo 1 ↔ Nodo 2

```
[consumidor-go@esc3-nodo2] <- pago.aprobado de productor-java@esc3-nodo1
[consumidor-python@esc3-nodo1] <- envio.entregado de productor-go@esc3-nodo2
```

Un evento publicado en Java (Nodo 1) lo consume Go (Nodo 2), y viceversa, sin conexión directa entre ellos

Evidencia: UI web de auditoría



- Consumidor Node.js con binding # recibe todos los eventos
- Los reenvía al navegador en tiempo real
- Columna «Nodo origen»: eventos de esc3-nodo1 y esc3-nodo2 en la misma vista

Comparación entre escenarios

Centralizado

+ Configuración mínima
+ Latencia muy baja
+ Logs en un solo lugar

– Cae el nodo → cae todo
– Limitado a una máquina

2 nodos

+ Broker aislado, sin competir por recursos
+ Si cae Nodo 1, los mensajes esperan en cola

– Requiere IP, puertos y firewall
– Un salto de red más

3 nodos

+ Carga repartida
+ Si cae un nodo de servicios, el otro sigue

– Más coordinación y logs repartidos
– Broker = punto único de falla

Retos técnicos

1

guest solo es local

Usuario admin propio para conexiones remotas

2

Arranque antes que el broker

Reintento de conexión + healthcheck/depends_on

3

NAT de Docker Desktop

Conexiones nombradas servicio@nodo e IP real con Get-NetTCPConnection

4

Firewall y PCs compartidos

Puerto 5672 en la LAN y script arreglar-docker.ps1

Conclusiones

- El mismo código funcionó en los 3 escenarios: la topología física queda fuera del código
- El exchange topic entrega un evento a varios consumidores sin que el productor los conozca
- Distribuir aísla recursos y tolera caídas de nodos, a cambio de más configuración de red
- El broker es el punto crítico: en producción se usaría un clúster RabbitMQ